Cryptographic Hash-Based File Integrity Verification System

PROJECT REPORT

BACHELOR OF TECHNOLOGY IN

COMPUTER SCIENCE AND ENGINEERING

By

Batch-7

G.Jyothsna(323103210063) **G.GeethaVyshnavi** (323103210080)

G.Vanitha (323103210070) **Hasini Lankalapalli**(323103210082)

Under the esteemed guidance of

Dr. N. Sharmili

Professor

CSE Department

Department of Computer Science and Engineering

GAYATRI VIDYA PARISHAD COLLEGE OF ENGINEERING FOR WOMEN

[Approved by AICTE NEW DELHI, Affiliated to Andhra University, Visakhapatnam]

[Accredited by National Board of Accreditation (NBA) for B. Tech. CSE, ECE & IT – valid from 2019-22 and 2022-25]

[Accredited by National Assessment and Accreditation Council (NAAC) with A Grade-Valid from 2022-2027]

Kommadi, Madhurawada, Visakhapatnam – 530048

2023 – 2027

GAYATRI VIDYA PARISHAD COLLEGE OF ENGINEERING FOR WOMEN

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

This is to certify that the Project report titled "Cryptographic Hash-Based File Integrity Verification System" is a bonafide work of following III B.Tech. –II Sem. students in the Department of Computer Science and Engineering, Gayatri Vidya Parishad College of Engineering for Women affiliated to Andhra University, Visakhapatnam during the academic year 2025-26, in fulfilment of the requirement for the award of the degree of Bachelor of Technology of this university.

G.Jyothsna(323103210063) **G.Geetha Vyshnavi** (323103210080)
G.Vanitha (323103210070) **Hasini Lankalapalli** (323103210082)

Signature of the Guide

Dr. N. Sharmili MTech., Ph.D.
Professor,
Dept. of CSE

Signature of the HOD

Dr. P. V. S. L. Jagadamba MTech, Ph.D
Professor,
Dept. of CSE

External Examiner

INDEX

S. NO	EXPERIMENT	PAGE NO
	ABSTRACT	
1.	SYSTEM REQUIREMENTS 1.1. Software Requirements 1.2 Hardware Requirements	1-2
2.	METHODOLOGY 2.1 System Architecture 2.2 Description, Algorithm	3-6
3.	IMPLEMENTATION 3.1Source code	7-8
4.	RESULTS 4.1 Output Screens	9-10
5.	CONCLUSION	11
6.	FUTURE SCOPE	12
7.	REFERENCES	13

ABSTRACT

In today's digital world, ensuring the integrity of data is a critical requirement in cybersecurity. Unauthorized modification of files can lead to data corruption, security breaches, and loss of trust. This project presents a File Integrity Verification System using the SHA-256 cryptographic hash function.

The system generates a fixed-length 256-bit hash value for any given file using a manually implemented SHA-256 algorithm. This hash uniquely represents the file content. Even a minor modification in the file results in a completely different hash value due to the avalanche effect.

The generated hash is stored in a database. During verification, the file is reprocessed to generate a new hash, which is then compared with the stored hash. If both hashes match, the file integrity is confirmed; otherwise, the file is identified as tampered.

This project demonstrates the practical application of cryptographic hashing in ensuring data integrity and security in file systems.

1.SYSTEM REQUIREMENTS

1.1. Software Requirements

1. SYSTEM REQUIREMENTS

1.1 Software Requirements

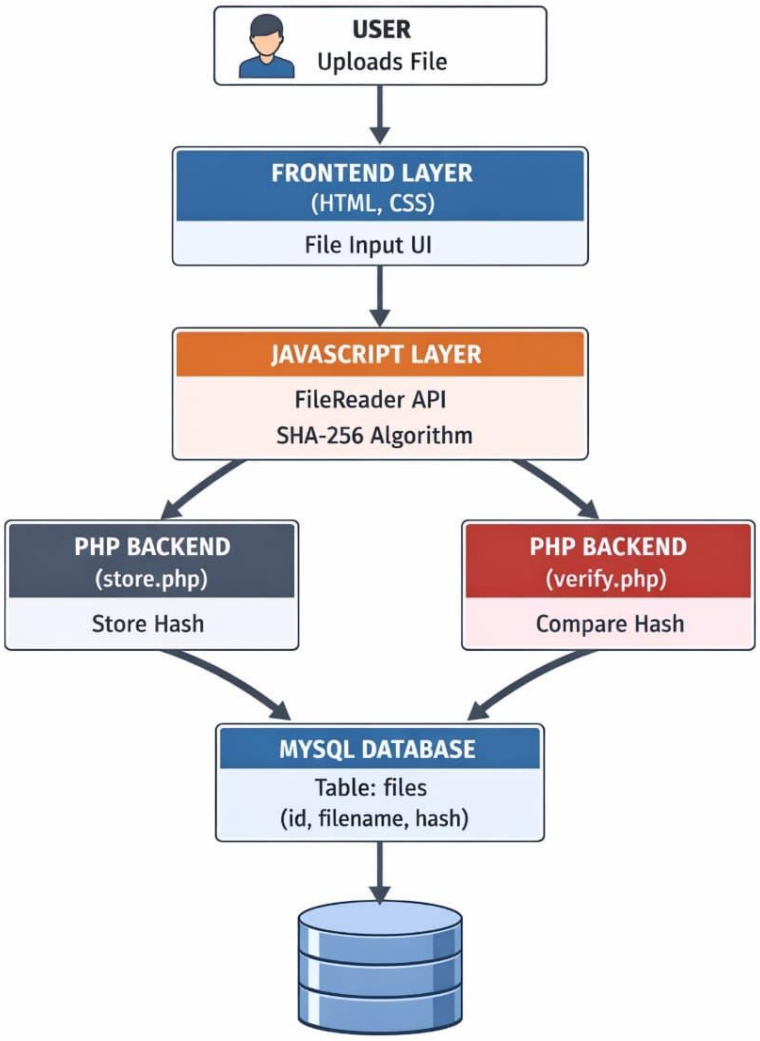
S.No	Software	Description
1	XAMPP	Provides Apache server and MySQL database
2	Visual Studio Code	Code editor for development
3	Web Browser	Chrome / Edge for execution
4	PHP	Backend scripting language
5	JavaScript	SHA-256 algorithm implementation
6	MySQL	Database for storing hash values
7	HTML & CSS	Frontend interface

1.2 Hardware Requirements

S.No	Component	Specification
1	Processor	Intel i3 or higher
2	RAM	Minimum 4 GB
3	Storage	10 GB free space
4	System	Laptop/Desktop
5	OS	Windows 10/11

2.METHODOLOGY

2.1 SYSTEM ARCHITECTURE



2.1 Components in Detail

1. Client (User Interface)

Role: Presentation layer — allows the user to interact with the system.

Technology: Python (Streamlit / Tkinter / CLI)

Functionality:

- Accepts user input (file upload or file path).
- Sends the selected file to the hashing module.
- Requests integrity verification.

- Displays generated hash and verification results.

2. Hash Generator (Integrity Module)

Role: Core processing layer — generates cryptographic hash values.

Technology: Python libraries (hashlib).

Functionality:

- Takes input file from client.
- Applies cryptographic hash function such as **SHA-256** or **MD5**.
- Produces a fixed-length hash value (digest).
- Ensures even small changes in file produce completely different hash.

3. Database / Storage (Hash Repository)

Role: Storage layer — stores original hash values securely.

Technology: File system / SQLite / Database

Functionality:

- Stores the original hash value of files.
- Maintains mapping between file name and hash value.
- Retrieves stored hash for verification.

4. Verification Module

Role: Comparison layer — verifies file integrity.

Technology: Python

Functionality:

- Recomputes hash of the file.
- Compares newly generated hash with stored hash.
- Determines whether file is modified or not.

Communication Flow (Step by Step)

1. Client → Hash Generator

- User uploads/selects a file.
- Client sends file data to hash generator.

2. Hash Generator → Storage

- Hash is generated using SHA-256/MD5.
- Hash value is stored in database as original reference.

3. Client → Verification Module

- User requests file verification.

4. Verification Module → Hash Generator

- Hash is recomputed for the same file.

5. Verification Module → Storage

- Stored hash value is retrieved.

6. Comparison Process

- New hash is compared with stored hash.

7. Result → Client

- If both match → File is Safe
- If different →File is Tampered
- Result is displayed to the user.

3. IMPLEMENTATION

3.1Source code

Index.html

```html
<!DOCTYPE html>
<html>
<head>
<title>File Integrity Checker</title>
<link rel="stylesheet" href="style.css">
</head>

<body>

<h2>File Integrity Checker</h2>

<input type="file" id="fileInput">

<br><br>

<button onclick="generateAndStore()">Generate & Store Hash</button>

<button onclick="checkIntegrity()">Check Integrity</button>

<p><b>Hash:</b></p>
<p id="hashOutput"></p>

<p id="result"></p>

<script src="sha256.js"></script>

</body>
</html>
```

Sha256.j

```javascript
function rightRotate(value, amount) {
    return (value >>> amount) | (value << (32 - amount));
}

function sha256(ascii) {

    var mathPow = Math.pow;
    var maxWord = mathPow(2, 32);
    var lengthProperty = 'length';

    var i, j;
    var result = '';

    var words = [];
    var asciiBitLength = ascii[lengthProperty] * 8;

    var hash = [], k = [];

    var primeCounter = 0;
    var isComposite = {};

    for (var candidate = 2; primeCounter < 64; candidate++) {
        if (!isComposite[candidate]) {
            for (i = 0; i < 313; i += candidate) {
                isComposite[i] = candidate;
            }
            hash[primeCounter] = (mathPow(candidate, .5) * maxWord) | 0;
            k[primeCounter++] = (mathPow(candidate, 1/3) * maxWord) | 0;
        }
    }

    ascii += '\x80';

    while (ascii[lengthProperty] % 64 - 56)
        ascii += '\x00';

    for (i = 0; i < ascii[lengthProperty]; i++) {
```

s

```
            b = a;
            a = (temp1 + temp2) | 0;
        }

        hash[0] = (hash[0] + a) | 0;
        hash[1] = (hash[1] + b) | 0;
        hash[2] = (hash[2] + c) | 0;
        hash[3] = (hash[3] + d) | 0;
        hash[4] = (hash[4] + e) | 0;
        hash[5] = (hash[5] + f) | 0;
        hash[6] = (hash[6] + g) | 0;
        hash[7] = (hash[7] + h) | 0;
    }

    for (i = 0; i < 8; i++) {
        for (j = 3; j + 1; j--) {
            var b = (hash[i] >> (j * 8)) & 255;
            result += ((b < 16) ? 0 : '') + b.toString(16);
        }
    }

    return result;
}
/* ================= CLEAN INTEGRITY SYSTEM ================= */

// Generate + Store
function generateAndStore(){

    let file = document.getElementById("fileInput").files[0];

    if(!file){
        alert("Upload a file first");
        return;
    }

    let reader = new FileReader();
```

```
    reader.onload = function(e){

        let content = e.target.result;
        let hash = sha256(content);

        document.getElementById("hashOutput").innerText = hash;

        // store in database
        let formData = new FormData();
        formData.append("filename", file.name);
        formData.append("hash", hash);

        fetch("store.php", {
            method: "POST",
            body: formData
        });

        document.getElementById("result").innerText =
            "✅ Hash generated & stored";
    };

    reader.readAsText(file);
}


// Check Integrity
function checkIntegrity(){

    let file = document.getElementById("fileInput").files[0];

    if(!file){
        alert("Upload file to verify");
        return;
    }

    let reader = new FileReader();
```

```
    reader.onload = function(e){

        let content = e.target.result;
        let hash = sha256(content);

        document.getElementById("hashOutput").innerText = hash;

        let formData = new FormData();
        formData.append("filename", file.name);
        formData.append("hash", hash);

        fetch("verify.php", {
            method: "POST",
            body: formData
        })
        .then(res => res.text())
        .then(data => {

            if(data === "safe"){
                document.getElementById("result").innerText =
                    " File is SAFE";
            }
            else{
                document.getElementById("result").innerText =
                    " File is TAMPERED";
            }
        });
    };

    reader.readAsText(file);
}
```

Store.php

```
<?php
$conn = new mysqli("localhost", "root", "", "sha256_db");

$filename = $_POST['filename'];
$hash = $_POST['hash'];

$conn->query("INSERT INTO files (filename, hash) VALUES ('$filename','$hash')");
?>
```

Style.css

```
body {
    font-family: Arial;
    text-align: center;
    margin-top: 50px;
}

button {
    padding: 10px 20px;
    margin: 5px;
}

#hashOutput {
    word-wrap: break-word;
    font-weight: bold;
}
```

Verify.php

```php
<?php
$conn = new mysqli("localhost", "root", "", "sha256_db");

$filename = $_POST['filename'];
$hash = $_POST['hash'];

$result = $conn->query("SELECT hash FROM files WHERE filename='$filename' ORDER BY id DESC LIMIT 1");

if($row = $result->fetch_assoc()){
    if($row['hash'] == $hash){
        echo "safe";
    } else {
        echo "tampered";
    }
}
?>
```

Database:

sha256_db.sql

```
CREATE DATABASE sha256_db;
USE sha256_db;
CREATE TABLE files (
    id INT AUTO_INCREMENT PRIMARY KEY,
    filename VARCHAR(255),
    hash TEXT
)
```

4. RESULTS

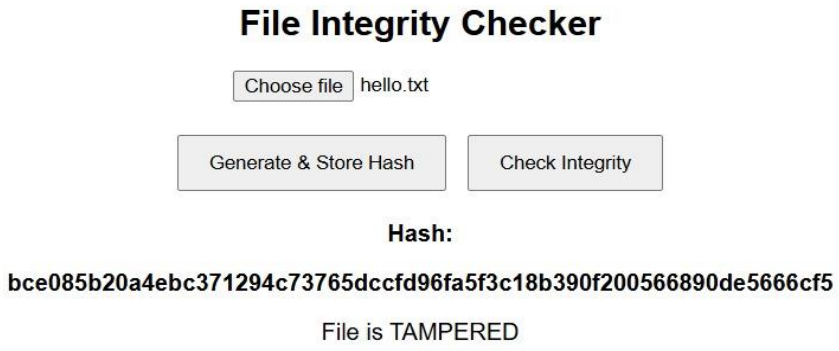
4.1 Output Screens

Choose a File to generate Hash.

Click on Generate & Store Hash to generate Hash.

Click on Check Integrity to check if the file is safe or tampered.

If File Content is changed, it displays..



5.CONCLUSION

The Cryptographic Hash-Based File Integrity Verification System provides a reliable and

efficient method to ensure data integrity. By using secure hash functions like SHA-256, he system can detect even the smallest changes in a file. This approach is widely used in cybersecurity, data transmission, and software distribution.

The system is simple, fast, and highly effective because:

- Hash functions generate unique outputs.
- Even a minor modification produces a completely different hash.
- It ensures trust and security in digital data.

Overall, this system plays a crucial role in protecting sensitive information and maintaining data authenticity in modern computing environments.

6.FUTURE SCOPE

The Cryptographic Hash-Based File Integrity Verification System can be further enhanced in several ways to improve its performance, usability, and security. As data security requirements continue to grow, the system can be extended with advanced features to make it more robust and suitable for real-world applications.

One major improvement is the support for **multiple file uploads**. Currently, the system may process one file at a time, but enabling batch processing will allow users to upload and verify multiple files simultaneously. This will significantly reduce processing time and improve efficiency, especially in environments where large volumes of data need to be verified regularly, such as in cloud storage systems or organizational data centers.

The **user interface (UI)** can also be enhanced to provide a more interactive and user-friendly experience. Features such as drag-and-drop file upload, progress indicators, real-time status updates, and clear visual feedback can make the system easier to use, even for non-technical users. A well-designed interface improves accessibility and encourages wider adoption of the system.

Another important enhancement is adding support for **multiple file formats**, including PDF documents, images, videos, and other file types. Although hash functions can technically work on any file type, optimizing the system to handle different formats efficiently will increase flexibility and usability. This makes the system more practical for real-world scenarios where users deal with diverse data types.

The system can also include a **file history tracking mechanism**, where details of previously uploaded files and their corresponding hash values are stored. This feature will allow users to monitor changes over time, verify past records, and maintain a log for auditing purposes. It is especially useful in environments where data integrity needs to be tracked continuously, such as in legal, financial, or research domains.

7.REFERENCES

Books Referred:

1. Cryptography And Network Security by Atul kahate, McGrawHill
2. Network Security Essentials- Applications and Standards by William Stallings, 3 rd Edition, Pearson

Online Resources:

1. GeeksforGeeks – SHA-256 Hash in Cryptography
   https://www.geeksforgeeks.org/sha-256-hash-in-cryptography/
2. GeeksforGeeks – What is File Integrity?
   https://www.geeksforgeeks.org/what-is-file-integrity/
3. W3Schools – PHP MySQL Tutorial
   https://www.w3schools.com/php/php_mysql_intro.asp
4. Tutorialspoint – Cryptography Basics
   https://www.tutorialspoint.com/cryptography/index.htm